KIPI: project structure

Contents of **kipi-kill-it-prove-it.zip**: a single-file key-value database that survives kill -9, its torture tool, and the required documents. Go, standard library only, no dependencies.

Build and run (from the repository root)

```
go build -o . ./cmd/kvdb ./cmd/torture
./torture --iterations 500 --seed 12345
go test ./...
```

On Windows the build produces kvdb.exe and torture.exe; run them as .\kvdb.exe and .\torture.exe.

Folder tree

kipi/	
├── go.mod	module kipi, Go 1.21+
├── .gitignore	ignores built binaries, torture-work/, *.kv
├── README.md	build, commands, serve protocol, on-disk format, recovery
├── ARCHITECTURE.md	diagram, 32-row proof table, 3 guarantees
├── BREAKING.md	crash points CP1–CP14, ordering, weaknesses
├── PROMPTS.md	AI prompt log (team must finish the TODO-TEAM entries)
├── **diagrams/**	
│ ├── architecture.mmd	diagram source (Mermaid)
│ └── architecture.png	rendered diagram
├── **src/kvdb/**	**THE ENGINE (library)**
│ ├── format.go	file header, record/marker encoding, CRC32C
│ ├── db.go	DB type, options, errors, Get, Close
│ ├── write.go	write path: Apply steps 1–12, disk-full guard
│ ├── recovery.go	Open(): lock, tmp cleanup, scan, truncate
│ ├── compact.go	compaction: tmp file → fsync → rename
│ ├── check.go	read-only integrity check
│ ├── platform_posix.go	Linux/macOS: flock, dir fsync, rename swap
│ ├── platform_windows.go	Windows: LockFileEx, close-rename-reopen
│ ├── sabotage_off.go	normal build: planted bugs compiled out
│ ├── sabotage_on.go	-tags sabotage: 4 planted bugs
│ └── kvdb_test.go	14 unit tests
├── **cmd/kvdb/**	**COMMAND-LINE WRAPPER → ./kvdb**
│ ├── main.go	open put get del batch dump check
│ └── serve.go	serve: stdin protocol, lanes, ACK
├── **cmd/torture/**	**TORTURE TOOL → ./torture**
│ ├── main.go	flags, iteration loop, coverage report
│ ├── session.go	starts serve, writers, SIGKILL, open/check/dump
│ ├── oracle.go	independent oracle + verify()
│ ├── gen.go	mode profiles, ops, self-describing values
│ └── scale.go	1M-key recovery time + file shrink proof
└── **tests/**	
└── e2e_test.go	builds binaries, kills serve by hand

■ engine ■ command line ■ torture tool ■ tests ■ documents

27 files, about 4,100 lines of Go. Built binaries and torture-work/ are not in the zip; they are created by the build and by running the tool.

What each file does

The names in the right-hand column are the functions or sections a judge is most likely to point at. The CP numbers are the `// CPn - CRASH HERE` comments in the code; they match the table in BREAKING.md.

src/kvdb/ (the engine)

File	Lines	What it does	Look for
`format.go`	163	On-disk layout: 16-byte header "KIPI", 12-byte record header, commit marker, CRC32C. Validates lengths before reading a payload.	`encodeHeader, appendRecord, encodeMarker, payloadLen, verifyPut`
`db.go`	204	The DB struct (index, fileSize, liveBytes, RWMutex, poisoned), Options, error codes, Get, sorted iteration, Stats, Close.	`Get, ForEachSorted, Close`
`write.go`	296	The single write path. Validate, disk-full guard, pwrite records, fsync, pwrite marker, fsync, index, ACK, auto-compaction. Rollback on errors.	`Apply (steps 1–12), CP1–CP7, spaceGuard, rollback`
`recovery.go`	291	open(): take lock first, create or clean tmp, scan the log, truncate the torn tail, ready.	`Open (phases 0–3), scan, createFile, CP8–CP10, CP14`
`compact.go`	140	Writes live records to db.kv.tmp, fsyncs, renames over db.kv, points the index at new offsets, fsyncs the directory.	`compactLocked (steps 1–8), CP11–CP13`
`check.go`	84	Read-only verification used by ./kvdb check. Reports an uncommitted tail without changing the file.	`Check`
`platform_posix.go`	51	Linux/macOS: flock, directory fsync, rename where the tmp descriptor becomes the database descriptor.	`lockFile, syncDir, installCompacted`
`platform_windows.go`	79	Windows: LockFileEx, no directory fsync, close both handles, rename, reopen.	`lockFile, installCompacted`
`sabotage_off.go / _on.go`	25	Normal build: sabotage = "" so planted bugs are dead code. -tags sabotage enables 4 bugs to prove the torture tool can fail.	`KVDB_SABOTAGE`

cmd/kvdb/ (command-line wrapper)

File	Lines	What it does	Look for
`main.go`	356	The 7 one-shot commands and the shared flags (--max-bytes, --hex, --trace, --compact-min-bytes, --step-delay-us).	`run, parseArgs, readBatchFile`
`serve.go`	299	Long-running mode. Text and hex dialects, BATCH n, lanes by op-id prefix, ACK printed under the write lock.	`serve, parseRequest, handle`

cmd/torture/ (torture tool)

File	Lines	What it does	Look for
`main.go`	442	Flags (--iterations --seed --mode), the iteration loop, the recovery-crash open kills, per-iteration line, coverage report, failure bundle.	`loop, account, fail, preload`
`session.go`	469	Runs serve as a child, 2+ concurrent writers, random-ms Process.Kill, ACK reader, open/check/dump helpers.	`runServe, handleLine, killOpen, runDump`
`oracle.go`	131	The oracle model and the check: dump must equal all ACKed ops, plus at most one whole in-flight op.	`model.apply, verify`
`gen.go`	243	Workload profile for each mode, key space, op generator, self-describing values.	`profileFor, gen, makeValue`
`scale.go`	196	1,000,000 keys: kill mid-batch, time recovery (< 2 s), verify every value, delete all, compact, check the file shrinks.	`runScale`

Tests			
File	**Lines**	**What it does**	**Look for**
src/kvdb/kvdb_test.go	533	Round trip, torn tail at every byte offset, uncommitted batch, F4, ordering, compaction, disk-full, lock, concurrency, readers.	go test ./src/...
tests/e2e_test.go	117	Builds the real binaries; kills serve 6 times like the judge; smoke-runs every torture mode.	go test ./tests/

Documents			
File	**Lines**	**What it does**	**Look for**
README.md	245	Build command, every command, serve protocol, on-disk format with a hex example, recovery in plain English, torture usage.	Judge minute 0–1
ARCHITECTURE.md	124	Diagram, all 32 scenario rows with code location and the run that verified it, the three guarantees.	Judge spot-checks 3 rows
BREAKING.md	77	Crash-point table CP1–CP14, the ordering argument, weaknesses, what failed, next 24 hours.	Crash-here answers
PROMPTS.md	370	Prompt log. Contains this chat verbatim; TODO-TEAM entries must be filled before submission.	–10 if missing
diagrams/	–	architecture.mmd (source) and architecture.png (rendered).	Open during the demo

How the parts connect

```
./torture ──starts──► ./kvdb serve ──calls──► src/kvdb (Apply/Get/Compact) ──► db.kv
    │                          │ prints "ACK id" after both fsyncs
    ├── records every ACK in its own oracle (cmd/torture/oracle.go) and torture-work/oracle.log
    ├── kills serve at a random millisecond (Process.Kill = SIGKILL / TerminateProcess)
    ├── runs ./kvdb open  →  src/kvdb/recovery.go Open (scan + truncate)
    ├── runs ./kvdb check →  src/kvdb/check.go Check
    └── runs ./kvdb dump  →  compares every key and value with the oracle
```

Files created at run time (not in the zip)

kvdb / kvdb.exe, torture / torture.exe	created by the build command in the repository root
<path> (e.g. db.kv)	the one authoritative data file
<path>.lock	holds the OS lock while a process has the database open; carries no data
<path>.tmp	exists only during creation or compaction; the next open deletes a leftover
torture-work/db.kv	the database the torture tool attacks (same file for all iterations)
torture-work/oracle.log	every ACKed op, written by the torture tool
torture-work/serve-stderr.log	stderr of every serve child
torture-work/failure/	only on failure: db-as-killed.kv, db-after-recovery.kv, oracle.log, report.txt

Mapping to the layout in the problem statement (section 10)

PDF expects	In this project
README.md, ARCHITECTURE.md, BREAKING.md, PROMPTS.md	same names, repository root
diagrams/	diagrams/
src/ (the engine)	src/kvdb/
torture/ (the torture tool)	cmd/torture/. A folder named torture/ in the root would block the ./torture binary on Linux/macOS, so the tool lives in cmd/.

tests/	tests/e2e_test.go; unit tests sit next to the engine in src/kvdb/kvdb_test.go (Go convention)
.git/	not in the zip: copy these files into your existing repository and commit in small steps

Suggested reading order for the team

1	`README.md, then format.go`	know the bytes on disk
2	`write.go Apply`	the write path and CP1–CP7
3	`recovery.go Open, scan`	recovery and CP8–CP10, CP14
4	`compact.go + platform_*.go`	compaction and CP11–CP13
5	`BREAKING.md`	check each crash point against the code
6	`cmd/torture/oracle.go, session.go`	how the oracle decides pass/fail